Detección de daños estructurales en nubes de puntos de dron mediante aprendizaje profundo: comparación de PointNet++ y Point Transformer V3 preentrenado sobre 3DAeroRelief

Jhonatan Velasco - 98251

[a] *Universidad de San Buenaventura Cali, Facultad de Ingeniería, Programa de Ingeniería de Sistemas, Cali, Valle del Cauca, Colombia*

Abstract

La inspección de estructuras civiles después de un desastre natural sigue dependiendo en gran medida de la observación visual, un proceso lento, subjetivo y riesgoso. Las nubes de puntos 3D obtenidas por fotogrametría con drones permiten automatizarla. Este trabajo estudia la segmentación semántica de edificaciones dañadas sobre el benchmark público 3DAeroRelief (huracán Ian, 2022), con su división oficial por áreas. Primero se identificaron y corrigieron dos inconsistencias no documentadas del dataset: la orientación vertical arbitraria de las reconstrucciones y una escala distinta en dos áreas. Luego se compararon, con un protocolo idéntico, cuatro modelos: PointNet++, Point Transformer V3 (PTv3) entrenado desde cero y PTv3 con codificadores preentrenados de forma autosupervisada (Sonata y Utonia). En el área de prueba, evaluada a la resolución original (30,9 millones de puntos), el mejor modelo, PTv3 + Utonia, obtiene un mIoU de 0,545 y un IoU de 0,749 en la clase edificación dañada, frente a 0,436 y 0,595 de PointNet++. PTv3 desde cero queda al nivel de PointNet++, lo que indica que, con pocos datos, el preentrenamiento importa más que la arquitectura; sin embargo, los modelos preentrenados muestran una variabilidad notable entre semillas. Finalmente, el modelo se integró en una aplicación web local que segmenta una nube y muestra el daño en 3D. El código, los modelos y la herramienta se entregan junto al documento.

Keywords: nubes de puntos, detección de daños estructurales, aprendizaje profundo, segmentación semántica 3D, PointNet++, Point Transformer V3, aprendizaje autosupervisado, evaluación post-desastre

1. Introducción

El deterioro y el daño de las estructuras civiles (edificaciones, puentes, túneles y construcciones patrimoniales) representan un riesgo constante para la seguridad pública y un costo significativo de mantenimiento. Tradicionalmente, este daño se identifica mediante inspección visual manual, un método lento y subjetivo que, en muchos casos, expone al personal a condiciones de riesgo, por ejemplo en escenarios posteriores a un desastre o en estructuras de difícil acceso [1].

En los últimos años, las tecnologías de captura 3D (la fotogrametría basada en imágenes de dron y el escaneo láser terrestre, TLS) han permitido representar estructuras completas como nubes de puntos densas, es decir, conjuntos masivos de coordenadas (x, y, z) que pueden incluir color o intensidad de retorno. Sobre esta representación, el aprendizaje profundo ha abierto la posibilidad de automatizar la detección, la segmentación y la cuantificación del daño estructural con una precisión superior a la de los métodos tradicionales basados en umbrales geométricos o en el análisis de imágenes 2D [1, 7].

En el primer avance se revisó la literatura y se propuso como conjunto de datos de trabajo el benchmark público 3DAeroRelief [6]; en el segundo se definió la metodología y se entrenó una primera línea base. Esta versión final del proyecto responde a la pregunta: *¿qué tan bien se puede identificar automáticamente una edificación dañada en una nube de puntos de dron, y cuánto ayudan las arquitecturas recientes y el preentrenamiento autosupervisado?* Para ello se plantean cuatro objetivos:

1. Revisar críticamente arquitecturas, conjuntos de datos y métricas, y relacionar cada trabajo con la propuesta (Sección 2).
2. Definir una metodología experimental clara y reproducible: dataset, preprocesamiento, modelos, entrenamiento y evaluación (Sección 3).
3. Comparar, bajo un mismo protocolo, una red clásica (PointNet++), una red reciente entrenada desde cero (Point Transformer V3) y la misma red preentrenada de forma autosupervisada, con varias semillas y a la resolución original de las nubes (Sección 4).
4. Llevar el modelo a una herramienta de software utilizable: una aplicación web local que segmenta una nube y muestra el daño en 3D (Secciones 3.7 y 4.5).

Las contribuciones son: (i) la detección y corrección de dos inconsistencias del dataset (orientación vertical arbitraria y escala distinta en dos áreas); (ii) una comparación con protocolo idéntico entre las tres familias de modelos; y (iii) el código y la herramienta, publicados junto

al documento. La Sección 5 discute e interpreta los resultados frente a trabajos similares, y la Sección 6 presenta las conclusiones y el trabajo futuro. El código, los notebooks ejecutados y los modelos entrenados están en el repositorio público `https://github.com/Jhodave31/deteccion-danos-nubes-puntos`.

2. Estado del arte

2.1. Enfoques híbridos: segmentación 2D combinada con reconstrucción 3D

Una primera línea de trabajo combina la segmentación semántica de imágenes con técnicas de reconstrucción 3D. En estos enfoques se segmentan imágenes capturadas por dron o por cámaras terrestres para localizar grietas o zonas de daño, y luego se proyecta esa información sobre una escena tridimensional reconstruida por fotogrametría o por SLAM. Así se aprovecha la madurez de los modelos de segmentación 2D y, al mismo tiempo, se incorpora el contexto espacial de la tercera dimensión.

Chen et al. [2] integran una red de segmentación de grietas a nivel de píxel con algoritmos de reconstrucción 3D para generar una nube de puntos semántica, sobre la cual aplican esqueletización y cuantificación geométrica. Obtienen errores relativos de longitud de apenas 1,3 % y errores de ancho de grieta inferiores a 0,15 mm. Chen et al. [3] fusionan nubes de puntos e imágenes 2D mediante un algoritmo de Otsu mejorado y alcanzan una precisión de 89,0 % en superficies de concreto. En túneles, Xu et al. [29] reconstruyen el revestimiento con SLAM y proyectan sobre el mapa 3D las máscaras de grieta obtenidas por una red 2D. Argumentan que los métodos puramente basados en imagen no capturan la curvatura de la sección del túnel. Ye et al. [15] proponen MCrack1300, un conjunto de 1300 imágenes anotadas de ladrillos y grietas, que combinan con el modelo Segment Anything (SAM) y fotogrametría monocular para estimar dimensiones de grieta con un error inferior al 10 % respecto al escaneo láser.

Estos enfoques alcanzan una precisión geométrica alta a escala de elemento, pero dependen de imágenes bien iluminadas y de una etapa de proyección 2D→3D que acumula errores de registro. Por eso esta propuesta opta por procesar directamente la nube.

2.2. Enfoques que operan directamente sobre la nube de puntos

Una segunda línea de investigación evita la conversión a imágenes y procesa directamente la geometría 3D. Lou, Meng y Zhou [5] presentan NL-3DCrack, una variante mejorada de PointNet++ para detectar grietas en 3D sin información de color, y alcanzan una exactitud de 94,2 % y un mIoU de 84,5 % sobre mampostería patrimonial. Bolourian et al. [26] proponen SNEPointNet++, que incorpora vectores normales y profundidad para segmentar grietas y desprendimientos (*spalls*) en puentes de concreto, y publican un conjunto de datos capturado en cuatro puentes. Fan et al. [27] extienden PointNet++ con un módulo de atención de dos ramas y la función de pérdida Poly Loss para extraer grietas de pavimento, atacando explícitamente el desbalance entre puntos de grieta y de fondo. En túneles ferroviarios escaneados con LiDAR, Liu et al. [28] combinan una red basada en grafos con un mecanismo de atención específico para grietas; reportan un IoU de 65,5 % para la clase grieta y señalan el desbalance de clases y las juntas constructivas como principales fuentes de error.

En el nivel de componentes estructurales, Jing et al. [11] proponen BridgeNet para la segmentación a gran escala de puentes de mampostería entrenada con un simulador sintético. Yang et al. [30] abordan la segmentación de componentes de puentes mediante aumento de datos sintéticos y aprendizaje métrico profundo sobre grafos. El mismo grupo de Jing et al. detecta grietas como anomalías geométricas, combinando histogramas de características de puntos rápidos (FPFH) con PatchCore sobre nubes sintéticas generadas por elementos finitos [8], y luego extiende el método a una representación multimodal [9].

En el patrimonio histórico, Loverdos y Sarhosis [14] clasifican bloques constructivos y detectan grietas a partir de modelos de reconstrucción 3D de mampostería. Dais et al. [10] combinan redes convolucionales con transferencia de aprendizaje y alcanzan una exactitud de 95,3 % en muros de mampostería. Herramientas integrales como Seg4D [12, 13] integran Point Transformer y algoritmos de auto-*machine learning* en CloudCompare para segmentar sistemas constructivos y analizar deformaciones.

2.3. Evaluación del daño post-desastre a escala de edificación y región

Cuando el objetivo es clasificar edificaciones completas tras un evento, y no grietas individuales, el problema cambia de escala. Zahs et al. [7] entrenan un *random forest* con nubes simuladas mediante escaneo láser virtual (VLS) y lo evalúan sobre nubes fotogramétricas reales del terremoto de L'Aquila (Italia, 2009). Alcanzan exactitudes globales entre 92,0 y 95,1 % y muestran que es posible transferir de datos sintéticos a reales. Shah, Shan y Li [31] reconstruyen regiones completas a partir de vuelos de dron, segmentan edificaciones con RANSAC y DBSCAN y estiman el estado de daño a partir de derivas residuales según FEMA P-58. En 2D, RescueNet [24] ofrece imágenes UAV de alta resolución tras el huracán Michael con etiquetas a nivel de píxel, incluidas varias clases de daño. Del mismo grupo proviene 3DAeroRelief [6], que lleva este tipo de anotación a nubes de puntos densas y es el benchmark en el que se basa este proyecto.

2.4. Arquitecturas de referencia para segmentación semántica 3D

PointNet [16] fue la primera red que consume directamente conjuntos de puntos desordenados mediante MLP compartidos y una función simétrica (*max pooling*). PointNet++ [17] añade una jerarquía de capas de abstracción

(*set abstraction*) que muestrean centros por punto más lejano (FPS), agrupan vecinos por radio y aplican PointNet localmente; luego interpola las características hacia los puntos originales. Qian et al. [18] (PointNeXt) mostraron que gran parte de la diferencia entre PointNet++ y redes más recientes se debe a las estrategias de entrenamiento (aumento de datos, optimizador) y no a la arquitectura. Ese resultado justifica usar PointNet++ como línea base con un entrenamiento moderno.

La familia Point Transformer introduce autoatención local: PTv1 [19] con atención vectorial, PTv2 [20] con atención vectorial agrupada y *pooling* por particiones, y PTv3 [21], que serializa la nube mediante curvas que recorren el espacio. Con esa serialización, PTv3 amplía el campo receptivo de 16 a 1024 puntos y es unas tres veces más rápido que PTv2. Fast Point Transformer [23] reduce el costo con atención ligera sobre vóxeles. OA-CNNs [22] muestra que las CNN dispersas con campos receptivos adaptativos pueden igualar a los transformers con menor latencia y memoria. Estas cinco redes son precisamente las evaluadas en 3DAeroRelief [6].

Una línea más reciente es el **preentrenamiento autosupervisado** de estas arquitecturas. Sonata [32] entrena el codificador de PTv3 sin etiquetas, mediante autodestilación entre vistas de una misma escena, y obtiene representaciones que se transfieren a tareas de segmentación con muy pocos datos etiquetados. Utonia [33] extiende la idea a un único codificador para nubes de dominios muy distintos (interiores, LiDAR automotor, escaneos aéreos y objetos). Ninguno de los modelos evaluados en 3DAeroRelief usa preentrenamiento, por lo que su efecto sobre la detección de daño post-desastre es una pregunta abierta que este trabajo aborda.

2.5. Conjuntos de datos y métricas

La Tabla 1 compara los conjuntos de datos más relevantes. Los datasets de grietas son mayoritariamente 2D (SDNET2018 [4], MCrack1300 [15]). Los 3D de defectos son pequeños o privados [26, 28], y los urbanos fotogramétricos a gran escala, como SensatUrban [25], no tienen clases de daño. En cuanto a métricas, en segmentación 3D predominan la exactitud global (OA), la exactitud media por clase (mAcc) y la intersección sobre unión media (mIoU) [25, 6, 21]. En detección de grietas es común reportar IoU o F1 de la clase de interés [28, 27], y en clasificación de edificaciones, la exactitud global [7]. Este proyecto reporta OA, mAcc, mIoU, IoU y F1 por clase para poder compararse con ambas comunidades.

2.6. Análisis comparativo y relación con la propuesta

La Tabla 2 resume los trabajos más cercanos e indica qué toma la propuesta de cada uno. Se pueden extraer cuatro observaciones:

- Las variantes de PointNet++ [5, 26, 27] siguen siendo competitivas en problemas de daño con pocos datos, y son el punto de partida natural para una línea base reproducible en una GPU de consumo.
- El desbalance de clases es el problema recurrente [27, 28, 26]. En 3DAeroRelief, la clase de interés compite con el fondo, por lo que la propuesta incorpora pesos por frecuencia en la función de pérdida.
- Los resultados reportados rara vez son comparables, porque cada trabajo usa su propio dataset. 3DAeroRelief es, hasta donde se pudo revisar, el único benchmark 3D público con daño post-desastre y resultados de referencia publicados [6], lo que permite una comparación cuantitativa directa.
- La generalización entre zonas es débil incluso para PTv3 (mIoU de 0,458 en el área de prueba [6]), lo que deja espacio para estudiar el preprocesamiento, el aumento de datos y la elección de arquitectura.

2.7. Vacíos de investigación

De la revisión se identifican los siguientes vacíos:

- **Escasez de datasets 3D abiertos y etiquetados:** la mayoría de los datasets públicos de grietas (SDNET2018 [4], MCrack1300 [15]) son de imágenes 2D, y los urbanos 3D [25] no incluyen clases de daño.
- **Dependencia de datos sintéticos:** varios trabajos recientes recurren a nubes generadas por simulación [7, 8, 11, 30], lo que deja abierta la pregunta de si los modelos se transfieren a condiciones reales de campo.
- **Clasificaciones de daño poco granulares y casos de estudio repetidos:** buena parte de los estudios post-desastre se limita a dos niveles (dañado / no dañado) y reutiliza los mismos eventos históricos.
- **Falta de comparaciones sobre un mismo benchmark:** los resultados se reportan sobre datasets propios, por lo que es difícil concluir qué arquitectura funciona mejor para el problema.

2.8. Tendencias actuales

Se observa una migración progresiva desde arquitecturas sobre imágenes 2D hacia modelos que operan directamente sobre la nube de puntos (PointNet++, Point Transformer v1–v3, CNN dispersas) [17, 21, 22]. La motivación es preservar toda la información espacial de la estructura. Al mismo tiempo, PointNeXt [18] y PTv3 [21] indican que la escala y las estrategias de entrenamiento pesan tanto como el diseño de la red. También se consolida el uso de datos sintéticos [7, 8] y aparecen benchmarks reales de bajo costo capturados con drones [6, 24], junto con herramientas integrales para profesionales de la inspección [12].

Tabla 1: Conjuntos de datos relacionados con la detección de daño estructural.

Dataset	Modalidad	Escena	Clases / anotación	Acceso
SDNET2018 [4]	Imagen 2D	Puentes, muros, pavimento	Grieta / no grieta (imagen)	Público
MCrack1300 [15]	Imagen 2D	Mampostería	Ladrillo, grieta (instancia)	Público
RescueNet [24]	Imagen UAV	Post-huracán	11 clases, incl. niveles de daño (píxel)	Público
SensatUrban [25]	Nube fotogr.	Ciudad (7,6 km^2)	13 clases urbanas, sin daño (punto)	Público
Bolourian et al. [26]	Nube LiDAR	4 puentes	Grieta, desprendimiento, sin defecto	Público
Zahs et al. [7]	Nube VLS + fotogr.	Post-sismo	Grados de daño (edificación)	Parcial
3DAeroRelief [6]	Nube fotogr. UAV	Post-huracán	5 clases, incl. edificación dañada (punto)	Público

Tabla 2: Análisis comparativo de trabajos relacionados con la propuesta.

Trabajo	Arquitectura	Datos	Resultado	Relación con la propuesta
Lou et al. [5]	PointNet++ mejorada	Mampostería, sin color	Acc. 94,2 %, mIoU 84,5 %	Respalda PointNet++ como base
Bolourian et al. [26]	SNEPointNet++	4 puentes (LiDAR)	Recall ≥92 %	Rasgos geométricos y desbalance
Fan et al. [27]	PointNet++ + atención	Pavimento	—	Pérdida para clases minoritarias
Liu et al. [28]	Grafo + atención	Túnel (LiDAR)	IoU grieta 65,5 %	Desbalance y falsos positivos
Zahs et al. [7]	Random forest	VLS → real	OA 92–95 %	Daño a escala de edificación
Shah et al. [31]	RANSAC + DBSCAN	UAV regional	Estado de daño FEMA	Escala regional, sin aprendizaje profundo
Le et al. [6]	PTv1/v2/v3, FPT, OA-CNNs	3DAeroRelief	mIoU 0,11–0,46	Benchmark y línea de comparación
Propuesta	PointNet++ (SSG) + pérdida ponderada	3DAeroRelief	Sección 4	—

3. Metodología

La Figura 1 resume el flujo experimental. Todo el código (preprocesamiento, modelos, entrenamiento, evaluación y herramienta) fue escrito en Python y PyTorch para este proyecto y está disponible en `https://github.com/Jhodave31/deteccion-danos-nubes-puntos`.

3.1. Selección y justificación del dataset

Se utiliza 3DAeroRelief [6], el primer benchmark 3D diseñado para evaluar el daño estructural post-desastre mediante segmentación semántica de nubes de puntos. El dataset se eligió por cinco razones:

1. Incluye explícitamente la clase *Building-Damage*, que corresponde a la variable objetivo del proyecto.
2. Son datos reales capturados en campo tras un desastre natural, a diferencia de gran parte de la literatura revisada, que usa datos sintéticos o privados [7, 8].
3. Ya fue evaluado con cinco modelos de referencia (PTv1, PTv2, PTv3, FPT y OA-CNNs) bajo una división fija, lo que permite comparar cuantitativamente los resultados del proyecto (Tabla 9).
4. Es abierto: el artículo se publica bajo licencia CC BY 4.0 y el repositorio oficial distribuye los datos bajo CC0, lo que permite usarlos y adaptarlos citando la fuente.
5. Aporta datos reales con etiquetas a nivel de punto, que es justamente el primer vacío identificado en la Sección 2.

3.2. Descripción del dataset

Las nubes se obtuvieron a partir de video aéreo capturado por drones (fotogramas de 1920×1080) sobre zonas afectadas por el huracán Ian (Florida, Estados Unidos, 2022). Se reconstruyeron con *Structure-from-Motion* y *Multi-View Stereo* (COLMAP) [6]. Las etiquetas se produjeron anotando manualmente uno de cada diez fotogramas en 2D y proyectando esas etiquetas a 3D por votación mayoritaria. Las nubes se reescalaron tomando como referencia una casa de un piso de unos 3 m de altura [6].

El dataset contiene 64 nubes de puntos distribuidas en 8 áreas geográficas, cada una con al menos una edificación. Se distribuye en formato `.ply` en dos archivos por nube: `pp<i>.ply` con coordenadas (x, y, z) y color RGB, y

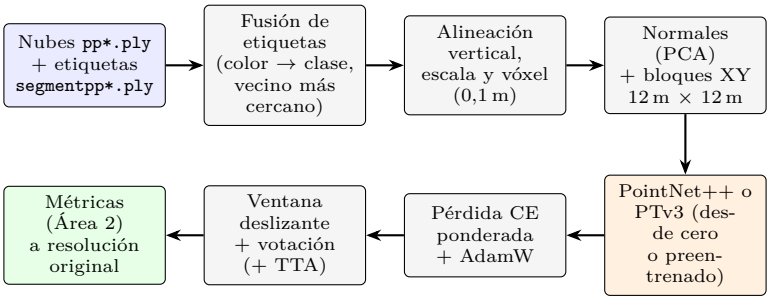


Figura 1: Flujo experimental propuesto, desde los archivos originales de 3DAeroRelief hasta las métricas de evaluación.

`segmentpp<i>.ply` con la etiqueta codificada como color. Cada punto pertenece a una de cinco clases: *Background*, *Building-Damage*, *Building-No-Damage*, *Road* y *Tree*. El paquete descargado ocupa unos 21 GB. La Tabla 3 presenta las estadísticas calculadas por el autor sobre los datos descargados, y la Figura 2 muestra la distribución de clases por área. En total hay 188,5 millones de puntos, con nubes de entre 0,31 y 16,6 millones (2,9 millones en promedio). Las nubes de las Áreas 2, 3 y 5 son mucho más densas porque provienen de capturas más cercanas. La distribución de clases está desbalanceada y varía mucho entre áreas. Destacan tres casos: la clase *Road* es casi inexistente en el Área 2 (0,6 % de los puntos de prueba), la clase *Building-No-Damage* no aparece en el Área 5, y en el Área 2 predomina la edificación dañada (37,8 %). Por eso la métrica IoU de la clase vía en el conjunto de prueba es muy sensible a pocos puntos y debe interpretarse con cautela.

3.3. Preprocesamiento

Fusión de etiquetas. El color de cada punto de `segmentpp<i>.ply` se asigna a la clase cuyo color de referencia es el más cercano (distancia euclidiana en RGB). Luego, cada punto de `pp<i>.ply` recibe la etiqueta de su vecino más cercano en la nube de etiquetas, buscado con un árbol k-d. Es el mismo procedimiento del script oficial del repositorio.

Alineación vertical. Durante la revisión de los datos se encontró que las nubes no están alineadas con la gravedad: COLMAP entrega cada reconstrucción con una orientación arbitraria (en varias nubes, el eje "vertical" original es y). Cada nube se rota a un sistema donde z es la normal del plano dominante (el eje de menor varianza según PCA, que corresponde al suelo). El sentido de z se fija con las normales de superficie, que COLMAP orienta hacia la cámara y por lo tanto deben apuntar hacia arriba (hacia el dron). Las 10 nubes que no traen normales usan como regla alternativa la asimetría positiva de la distribución de alturas. Ninguna de estas reglas usa etiquetas. Como verificación a posteriori, en 62 de las 64 nubes las clases edificación y árbol quedan por encima de vía y fondo. Las dos excepciones (Área 1/pp7 sin normales y Área 3/pp11) pertenecen al conjunto de entrenamiento.

Armonización de escala. El artículo indica que las nubes se reescalaron con una casa de un piso de unos 3 m como referencia [6]. Sin embargo, en las Áreas 4 y 6 las edificaciones aparecen entre 3 y 3,5 veces más altas que en las demás áreas. Para cada área de entrenamiento y validación se estimó la altura típica de las edificaciones sobre el suelo (mediana de z de los puntos de edificación menos el percentil 5 de z) y se corrigieron las áreas que se desviaban más de 1,5 veces de la mediana de las demás: factor 0,285 para el Área 4 y 0,354 para el Área 6. El Área 2 (prueba) no se usa en este cálculo ni se modifica.

Submuestreo por vóxel. Para homogeneizar la densidad (que varía entre capturas) y reducir el costo, se conserva un punto aleatorio por vóxel de 0,1 m de lado. Así se pasa de 188,5 a 18,8 millones de puntos (Tabla 3).

Normales de superficie. Las normales originales solo existen en 54 de las 64 nubes, así que se recalculan para todas de forma uniforme sobre los puntos submuestreados: para cada punto se toma el vector propio de menor valor propio de la covarianza de sus 16 vecinos más cercanos (PCA local) y se orienta hacia $+z$. Las normales describen la orientación local de la superficie, que cambia de forma irregular en un techo con tejas faltantes o escombros.

Bloques de entrada. Como una nube completa no cabe en la red, durante el entrenamiento se extraen bloques columnares de 12 m × 12 m en el plano XY, centrados en un punto elegido al azar. Para PointNet++ se muestrean 4096 puntos por bloque y las coordenadas se dividen por el lado del bloque, de modo que los radios de agrupación son relativos al bloque. PTv3 admite un número variable de puntos, así que recibe todos los puntos del bloque (hasta 16 000) en coordenadas métricas.

Rasgos de entrada. En PointNet++ cada punto tiene 7 canales: color RGB normalizado a $[0, 1]$, altura relativa dentro de la nube y posición (x, y, z) normalizada respecto a la caja envolvente de la nube. En PTv3 se usan los 9 canales que esperan los codificadores preentrenados: coordenadas locales, color RGB en $[0, 1]$ y normal de superficie.

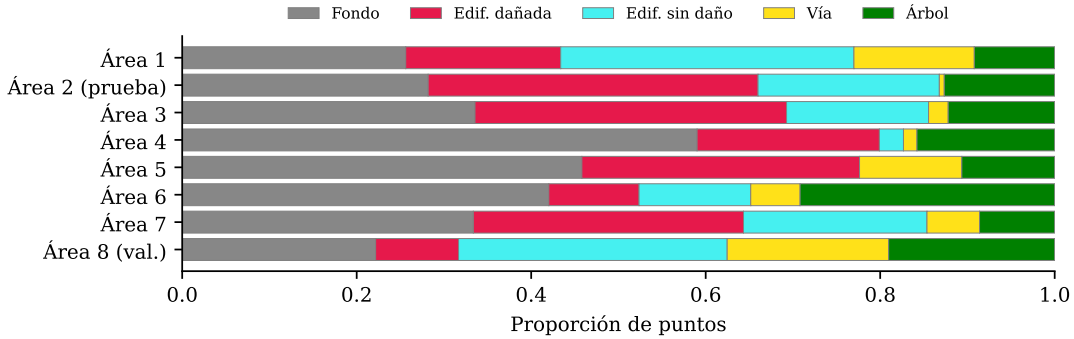


Figura 2: Proporción de puntos por clase en cada área tras el submuestreo. La variación entre áreas, por ejemplo la casi ausencia de vías en el Área 2, explica parte de la dificultad de generalizar entre zonas.

Tabla 3: Estadísticas de 3DAeroRelief calculadas por el autor. "Puntos orig.": puntos en los archivos originales; "Tras vóxel": puntos tras la alineación, la armonización de escala y el submuestreo a 0,1 m (Sección 3.3). Las cinco últimas columnas son el porcentaje de puntos de cada clase tras el submuestreo: fondo (Fon.), edificación dañada (E. D.), edificación sin daño (E. S.), vía y árbol.

Área	Uso	Nubes	Puntos orig. (M)	Tras vóxel (M)	Fon. %	E. D. %	E. S. %	Vía %	Árbol %
Área 1	Entrenamiento	7	5,7	2,5	25,6	17,8	33,6	13,8	9,2
Área 2	Prueba	8	30,9	1,7	28,2	37,8	20,8	0,6	12,7
Área 3	Entrenamiento	13	90,4	2,9	33,6	35,7	16,3	2,2	12,2
Área 4	Entrenamiento	4	7,8	2,4	59,0	20,9	2,8	1,5	15,8
Área 5	Entrenamiento	4	28,4	1,3	45,8	31,8	0,0	11,7	10,6
Área 6	Entrenamiento	12	15,3	5,4	42,0	10,3	12,8	5,7	29,2
Área 7	Entrenamiento	10	5,2	1,6	33,4	31,0	21,0	6,0	8,6
Área 8	Validación	6	4,8	0,9	22,2	9,5	30,8	18,5	19,0
Total	—	64	188,5	18,8	37,9	22,2	16,3	6,4	17,1

Aumento de datos. En entrenamiento se aplican rotación aleatoria alrededor de z, escalado uniforme en [0,9; 1,1], reflexión en x, ruido gaussiano en las coordenadas ($\sigma = 0{,}002$), variación de brillo y contraste, y eliminación del color con probabilidad 0,2. Esta última técnica mejora la generalización de PointNet++ según Qian et al. [18].

3.4. Modelos

3.4.1. PointNet++ (línea base)

Como línea base se implementó PointNet++ en su variante de agrupación a escala única (SSG) [17]. Se eligió por cuatro razones: (i) es la base de varios de los trabajos de daño revisados [5, 26, 27]; (ii) según PointNeXt [18], con un entrenamiento moderno compite con arquitecturas más recientes; (iii) cabe en una GPU de consumo de 8 GB; y (iv) no está entre los modelos evaluados en el benchmark, así que aporta un punto de comparación nuevo. La Figura 3 muestra el esquema y la Tabla 4 el detalle de las capas.

Cada capa de abstracción (SA) hace tres cosas: (1) selecciona N_s centros por muestreo del punto más lejano, (2) agrupa hasta K vecinos dentro de un radio r alrededor de cada centro y (3) aplica un MLP compartido sobre las coordenadas relativas y los rasgos de los vecinos, seguido de *max pooling*. Cada capa de propagación (FP) interpola las características del nivel más escaso hacia el más denso mediante ponderación por distancia inversa de los 3 vecinos más cercanos. Además, las concatena con las características del mismo nivel del codificador (conexión de salto). Todos los operadores (FPS, *ball query* e interpolación) se implementaron con operaciones tensoriales estándar de PyTorch, sin extensiones CUDA, para facilitar la instalación en Windows.

Tabla 4: Detalle de capas de la red. N_s: centros muestreados por FPS; r: radio de agrupación (en unidades normalizadas por el lado del bloque); K: vecinos por grupo. Cada capa del MLP es Conv 1×1 + BatchNorm + ReLU.

Capa	Tipo	N_s	r / K	MLP (canales)
SA1	Set abstraction	1024	0,1 / 32	$(3+7+3) \to [32, 32, 64]$
SA2	Set abstraction	256	0,2 / 32	$[64, 64, 128]$
SA3	Set abstraction	64	0,4 / 32	$[128, 128, 256]$
SA4	Set abstraction	16	0,8 / 32	$[256, 256, 512]$
FP4	Feature propagation	64	3-NN	$768 \to [256, 256]$
FP3	Feature propagation	256	3-NN	$384 \to [256, 256]$
FP2	Feature propagation	1024	3-NN	$320 \to [256, 128]$
FP1	Feature propagation	4096	3-NN	$135 \to [128, 128, 128]$
Cabeza	Conv 1×1	4096	—	$128 \to 128 \to$ Dropout(0,
Parámetros entrenables: 964 229				

3.4.2. Point Transformer V3 (PTv3)

PTv3 [21] reemplaza la búsqueda de vecinos por radio de PointNet++ por una *serialización*: los puntos se ordenan a lo largo de curvas que recorren el espacio (curvas

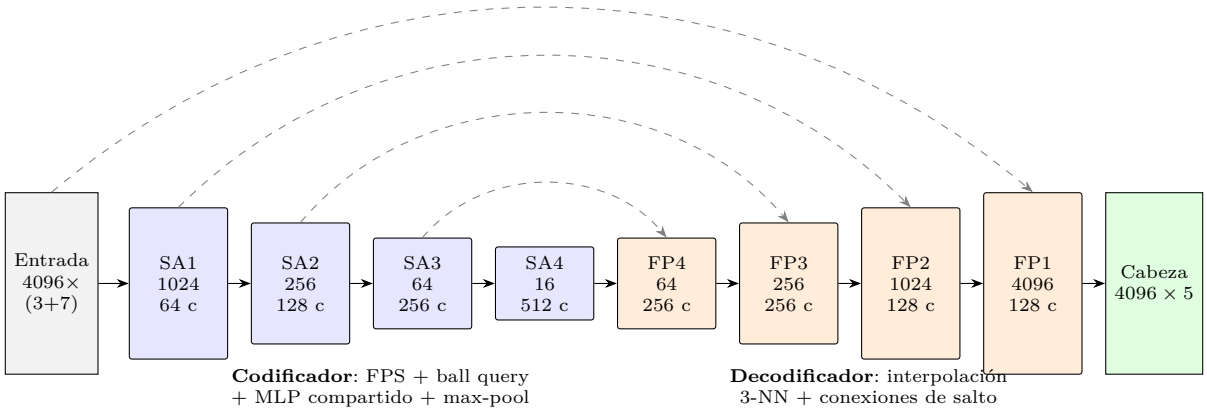


Figura 3: Arquitectura PointNet++ (SSG) para segmentación semántica. Bajo cada bloque se indica el número de puntos y de canales (c) de salida. Las flechas punteadas son conexiones de salto. La cabeza produce una probabilidad para cada una de las 5 clases en cada punto.

de Morton "z" y de Hilbert, y sus versiones transpuestas), y la atención se calcula dentro de parches consecutivos de 1024 puntos de esa secuencia. Así, cada punto atiende a un vecindario mucho más amplio (1024 puntos frente a los 32 de cada grupo de PointNet++), que puede abarcar una casa completa, y el costo sigue siendo bajo. Entre bloques se alterna el orden de serialización, de modo que los parches cambian de forma y la información fluye en todas las direcciones.

Se usa la implementación oficial de Pointcept, distribuida como el paquete `point-transformer-v3`. Esta versión ya no necesita las bibliotecas `spconv` ni FlashAttention y funciona en Windows con la atención estándar de PyTorch (SDPA). La Tabla 5 resume la configuración, que es la misma de la publicación original. La red tiene un codificador de 5 niveles (profundidades 2, 2, 2, 6 y 2; canales de 32 a 512) que reduce la resolución a la mitad entre niveles, y un decodificador de 4 niveles con conexiones de salto.

3.4.3. PTv3 con preentrenamiento autosupervisado

Para estudiar el efecto del preentrenamiento se parte del codificador `sonata_small`, preentrenado con Sonata [32] sobre miles de escenas 3D sin etiquetas, y del codificador `utonia_tiny` de Utonia [33], preentrenado sobre nubes de muchos dominios, incluidos escaneos aéreos. Estos codificadores no tienen decodificador. Las características de sus cinco niveles se llevan a la resolución original y se concatenan (992 canales en Sonata, 486 en Utonia), y sobre ellas se entrena una cabeza MLP ($\rightarrow 256 \rightarrow$ BatchNorm, ReLU, Dropout 0,3 $\rightarrow 5$). Todo el modelo se ajusta (*fine-tuning*): la cabeza con la tasa de aprendizaje base y el codificador con una tasa diez veces menor, para no destruir lo aprendido en el preentrenamiento.

Los codificadores preentrenados esperan una escala determinada: Sonata trabaja con celdas de 2 cm en escenas interiores y Utonia con celdas de 1 cm tras multiplicar las coordenadas por un factor. Por eso las coordenadas se escalan (por 0,2 en Sonata y 0,1 en Utonia) de modo que cada celda de su grilla corresponda a un vóxel de 0,1 m del preprocesamiento, y no se pierda ningún punto.

3.5. Protocolo de entrenamiento

Se respeta la división cruzada por áreas del benchmark [6]: el Área 2 (8 nubes) es el conjunto de prueba y no se usa en ningún momento del entrenamiento ni de la selección del modelo. Como el benchmark no define un conjunto de validación, se reservó el Área 8 para seleccionar el mejor modelo, y el entrenamiento se hace con las Áreas 1, 3, 4, 5, 6 y 7. Por lo tanto, el modelo ve algo menos de datos que los modelos de referencia, una decisión conservadora que favorece a estos últimos en la comparación.

Para mitigar el desbalance de clases se usa entropía cruzada ponderada, con peso $w_c \propto 1/\sqrt{f_c}$, donde f_c es la frecuencia de la clase c en el entrenamiento, normalizado para que el peso medio sea 1. La Tabla 6 resume los hiperparámetros de cada modelo. Para que la comparación sea justa, todos comparten los mismos bloques, la misma división de áreas, el mismo aumento de datos, la misma pérdida y el mismo criterio de selección del modelo (mejor mIoU en el Área 8).

3.6. Protocolo de evaluación

La inferencia se hace sobre cada nube completa del Área 2 con una ventana deslizante de 12 m $\times$ 12 m y paso igual a la mitad del lado. Todos los puntos de cada ventana se procesan en grupos de 4096. Como las ventanas se solapan, cada punto recibe varias predicciones; se suman las probabilidades y se toma la clase de mayor probabilidad acumulada. Con la matriz de confusión acumulada sobre

Tabla 5: Configuración de los modelos PTv3. En todos, las profundidades del codificador son (2, 2, 2, 6, 2), la atención se calcula en parches de 1024 puntos con cuatro órdenes de serialización, y el *drop path* es 0,3.

	PTv3 desde cero	Sonata (`sonata_small`)	Utonia (`utonia_tiny`)
Pesos iniciales	Aleatorios	Preentrenados	Preentrenados
Canales del codificador	32–512	32–512	18–216
Decodificador	Sí (64–256)	No (concatenación)	No (concatenación)
Cabeza	Lineal	MLP 992–256–5	MLP 486–256–5
Celda de grilla	0,1 m	0,02 (escala 0,2)	0,01 (escala 0,1)
Parámetros	46,2 M	38,9 M	9,5 M

Tabla 6: Hiperparámetros y entorno de entrenamiento.

Parámetro	PointNet++	PTv3 (desde cero / preentrenado)
Optimizador	AdamW, *weight decay* 10^{-4}	AdamW, *weight decay* 0,05
Tasa de aprendizaje	10^{-3} máx.	$2 \cdot 10^{-3}$ máx.; codificador preentrenado: $2 \cdot 10^{-4}$
Política	*One-cycle* (calentamiento 5 % + coseno)	Igual
Lote	16 bloques × 4096 puntos	4 bloques completos (hasta 16 000 puntos c/u)
Duración	40 épocas × 400 iteraciones	30 épocas × 400 iteraciones
Pérdida	Entropía cruzada ponderada ($w_c \propto f_c^{-1/2}$)	
Precisión	Mixta (FP16)	Mixta (BF16), recorte de gradiente 10
Validación	Cada 4 épocas (Área 8)	Cada 3 épocas (Área 8)
Semillas	0, 1 y 2	
Hardware	GPU NVIDIA RTX 4060 Ti (8 GB), CPU AMD Ryzen 5 5600, 32 GB RAM	
Software	Windows 11, Python 3.12, PyTorch 2.14 (CUDA 12.6), `point-transformer-v3` 0.1.2	

todo el conjunto de prueba se calculan:

$$\mathrm{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \qquad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c, \quad (1)$$

$$\mathrm{Acc}_c = \frac{TP_c}{TP_c + FN_c}, \qquad \mathrm{mAcc} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{Acc}_c, \quad (2)$$

$$F1_c = \frac{2\,TP_c}{2\,TP_c + FP_c + FN_c}, \qquad \mathrm{OA} = \frac{\sum_c TP_c}{N}, \quad (3)$$

donde $C = 5$ es el número de clases y N el número de puntos evaluados. mIoU y mAcc se definen igual que en el benchmark [6], de modo que los resultados son directamente comparables.

Resolución original. Los modelos predicen sobre los puntos submuestreados a 0,1 m. Para reportar el desempeño sobre la nube completa (30,9 millones de puntos en el Área 2), cada punto original recibe la predicción de su vecino submuestreado más cercano, aplicando antes la misma alineación vertical del preprocesamiento.

Aumento en prueba (TTA). Opcionalmente, cada nube se predice ocho veces (cuatro rotaciones de 90° alrededor de z, con y sin reflexión) y se promedian las probabilidades.

Semillas. Según el presupuesto de cómputo, cada configuración se entrenó con hasta tres semillas (0, 1 y 2), priorizando los modelos preentrenados; el número exacto se indica en la Tabla 7. Se reporta la media ± la desviación estándar, para distinguir diferencias reales de variaciones aleatorias del entrenamiento.

3.7. Implementación, reproducibilidad y herramienta

El proyecto se organizó como un producto de software reproducible. El repositorio público (`https://github.com/Jhodave31/deteccion-danos-nubes-puntos`) contiene: los módulos de preprocesamiento, modelos, entrenamiento y evaluación (`code/`); un guion que ejecuta todos los experimentos en cadena (`run_all.sh`); tres notebooks ejecutados con sus salidas (`notebooks/`: datos y preprocesamiento, modelos y entrenamiento, y evaluación); las métricas de cada corrida (`runs/`); y, como *release*, los modelos entrenados. El notebook de evaluación vuelve a ejecutar un modelo entrenado sobre el área de prueba y reproduce las métricas reportadas; todas las tablas y figuras de la Sección 4 se generan con el mismo código (`aggregate.py`) a partir de esas métricas.

Además, el modelo se integró en una aplicación web local para que un usuario sin conocimientos de aprendizaje profundo pueda analizar una nube. Sus requisitos funcionales son: seleccionar una nube del dataset o cargar un `.ply` propio, segmentarla con el mismo preprocesamiento del experimento, visualizarla en 3D (coloreada por predicción, color real, etiqueta real o mapa de errores) y resumir la proporción de cada clase. Sus requisitos no funcionales son funcionar sin conexión, responder en segundos y no requerir instalación adicional. La arquitectura es cliente-servidor local (Figura 4): un servidor en Python (biblio-

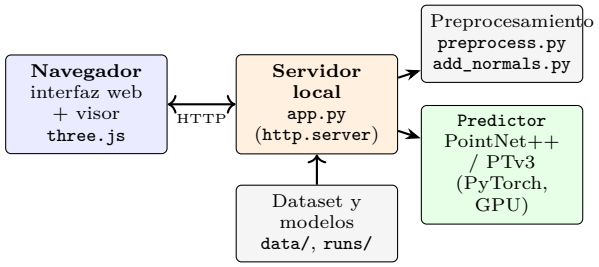


Figura 4: Arquitectura de la herramienta. Todo se ejecuta en el computador del usuario, sin servicios externos.

teca estándar `http.server`) *reutiliza* los módulos del experimento mediante una clase `Predictor`, que carga cualquiera de los modelos entrenados a partir de su carpeta de resultados; el cliente es una página web con `three.js`, incluida localmente, que dibuja hasta 700 000 puntos de forma interactiva.

4. Resultados

Se entrenaron 4 configuraciones con el protocolo de la Sección 3 (semillas: PointNet++: 3; PTv3 desde cero: 1; PTv3 + Sonata: 3; PTv3 + Utonia: 3). Cada modelo se seleccionó por su mIoU en el Área 8, y el Área 2 se evaluó una sola vez con el modelo elegido.

4.1. Desempeño en el área de prueba

La Tabla 7 y la Figura 5 resumen el desempeño en el área de prueba. Ordenados por mIoU medio a la resolución original, los modelos quedan así: PTv3 + Utonia (0,545 ± 0,030), PTv3 + Sonata (0,505 ± 0,045), PTv3 desde cero (0,447), PointNet++ (0,436 ± 0,033). El mejor modelo, PTv3 + Utonia, obtiene un IoU de 0,749 ± 0,054 en la clase objetivo, edificación dañada, frente a 0,595 ± 0,092 de PointNet++. Los modelos preentrenados mejoran sobre todo edificación sin daño, árbol y, en el caso de Utonia, vía. El aumento en prueba (TTA) cambia poco el mIoU (diferencia con TTA menos sin TTA, sobre puntos submuestreados: PointNet++: +0,001, PTv3 desde cero: +0,020, PTv3 + Sonata: +0,005, PTv3 + Utonia: −0,009). Las métricas a la resolución original difieren de las submuestreadas, porque las nubes densas de capturas cercanas pesan más; por ejemplo, pp4 tiene 12,6 millones de puntos y está dominada por una edificación dañada.

4.2. Entrenamiento y variabilidad entre semillas

La Figura 6 muestra que los modelos preentrenados alcanzan una validación alta en pocas épocas y luego oscilan, mientras la exactitud de entrenamiento sigue subiendo (sobreajuste). PTv3 desde cero aprende más despacio y aún mejoraba al terminar las 30 épocas. La Tabla 8 muestra que el mIoU de prueba cambia de forma notable con la semilla (PointNet++, de 0,403 a 0,469; PTv3 + Sonata, de 0,453 a 0,534; PTv3 + Utonia, de 0,517 a 0,577), y que semillas con una validación casi igual obtienen resultados de prueba distintos.

4.3. Comparación con el benchmark publicado

La Tabla 9 sitúa los resultados junto a los publicados por los autores del dataset sobre la misma área de prueba. El mejor modelo publicado, PTv3, obtiene un mIoU de 0,458, un mAcc de 0,551 y un IoU de edificación dañada de 0,761. PTv3 + Utonia obtiene 0,545, 0,737 y 0,749, respectivamente.

4.4. Resultados cualitativos y errores

En la nube pp6 (Figura 7), PointNet++ clasifica como *sin daño* buena parte del techo dañado, mientras que PTv3 + Utonia lo reconoce casi completo. En la matriz de confusión (Figura 8), el 88 % de los puntos de edificación dañada se clasifica correctamente. A la vez, el 33 % de los de edificación sin daño se marca como dañada, y el 58 % de la vía se confunde con fondo. Por nube (Tabla 10), PTv3 + Utonia mejora a PointNet++ en 5 de las 8 nubes y empeora en 3.

4.5. Herramienta en funcionamiento

La Figura 9 muestra la herramienta analizando una nube de prueba con el modelo PTv3 + Utonia. En la GPU, una nube típica del Área 2 (pp1, 0,55 millones de puntos) se preprocesa en 1,6 s y se segmenta en 4,2 s. En las nubes más densas (por ejemplo pp6, 5,0 millones de puntos) la segmentación sigue tomando unos 2 s, pero el preprocesamiento (lectura, alineación, submuestreo y normales) toma alrededor de 3 min. Por eso los análisis de las nubes del dataset se guardan en disco y la segunda vez se abren de inmediato. El modelo que usa la herramienta se indica en `demo/config.json` (actualmente, `runs/utonia_tiny_s2`). Se verificó que reproduce las predicciones de la evaluación (el acierto mostrado coincide con el del guion de evaluación sobre los mismos puntos), y se probaron los cuatro modos de vista, la carga de archivos propios y la ejecución sin conexión.

5. Discusión

5.1. Arquitectura frente a preentrenamiento

El hallazgo central es que, con los datos disponibles, la arquitectura por sí sola no basta: PTv3 entrenado desde cero, con 46 veces más parámetros, queda al nivel de PointNet++ (mIoU 0,447 frente a 0,436 ± 0,033). Lo que marca la diferencia es partir de representaciones aprendidas. Los dos modelos preentrenados superan a los dos entrenados desde cero en mIoU medio, y la diferencia entre ambos grupos es mayor que la variación entre semillas. Entre los dos preentrenados, Utonia rinde más que Sonata (0,545 frente a 0,505), aunque tiene 4,1 veces menos parámetros. La diferencia más clara está en la clase vía (0,222 frente a 0,068), una superficie exterior plana que Sonata, preentrenado con escenas interiores, no ha visto. Utonia sí incluye escaneos aéreos y LiDAR exterior. Esto sugiere que el dominio de los datos de preentrenamiento

Tabla 7: Resultados en el área de prueba (Área 2). “Vóxel”: puntos submuestreados a 0,1 m; “original”: los 30,9 millones de puntos de las nubes originales. Media ± desviación estándar sobre las semillas indicadas (sin desviación cuando hay una sola).

		mIoU (vóxel)		Resolución original, con TTA			
Modelo	Semillas	sin TTA	con TTA	mIoU	OA	mAcc	IoU dañada
PointNet++	3	0,461 ± 0,023	0,461 ± 0,023	0,436 ± 0,033	0,752 ± 0,055	0,606 ± 0,046	0,595 ± 0,092
PTv3 desde cero	1	0,449	0,469	0,447	0,778	0,637	0,731
PTv3 + Sonata	3	0,469 ± 0,064	0,474 ± 0,053	0,505 ± 0,045	0,824 ± 0,049	0,691 ± 0,018	0,745 ± 0,071
PTv3 + Utonia	3	0,523 ± 0,040	0,514 ± 0,049	0,545 ± 0,030	0,840 ± 0,028	0,737 ± 0,020	0,749 ± 0,054

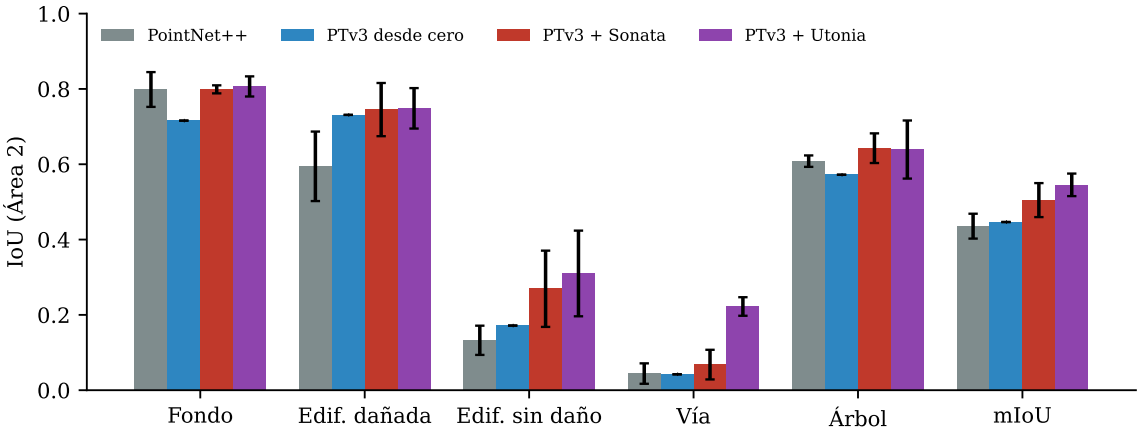


Figura 5: IoU por clase en el Área 2 (resolución original, con TTA). Las barras de error indican la desviación estándar entre semillas.

Tabla 8: mIoU de validación del modelo seleccionado (Área 8) y de prueba (Área 2, resolución original) para las semillas 0 / 1 / 2.

Modelo	mIoU validación	mIoU prueba
PointNet++	0,539 / 0,532 / 0,516	0,469 / 0,434 / 0,403
PTv3 + Sonata	0,552 / 0,553 / 0,552	0,528 / 0,453 / 0,534
PTv3 + Utonia	0,532 / 0,554 / 0,578	0,577 / 0,517 / 0,542

importa tanto como la cantidad de parámetros. Este resultado coincide con dos observaciones de la literatura. Por un lado, PointNeXt [18] mostró que las estrategias de entrenamiento pesan tanto como el diseño de la red. Por otro, en el benchmark original [6], los modelos entrenados desde cero generalizan mal entre áreas. Con solo 50 nubes de entrenamiento de seis zonas, un transformer grande no tiene datos suficientes para aprender desde cero la geometría de techos, muros y vegetación. Un codificador preentrenado ya trae esa geometría y solo debe aprender a separar las clases de este problema.

5.2. Comparación con trabajos similares

Frente al benchmark (Tabla 9), PTv3 + Utonia supera al mejor modelo publicado en mIoU (0,545 frente a 0,458) y en mAcc, y queda al mismo nivel en la clase edificación dañada (0,749 frente a 0,761). La comparación es **indicativa**: aquí se corrigieron la orientación vertical y la escala de las nubes (Sección 3.3), algo que los modelos de referencia no reportan, y los modelos se entrenaron sin el Área 8. La comparación entre los modelos de este trabajo, en cambio, sí es justa, porque todos comparten exactamente el mismo protocolo.

Frente a otros trabajos de daño estructural, las cifras de este proyecto son menores en valor absoluto, pero la tarea es más difícil. NL-3DCrack [5] alcanza un mIoU de 84,5 % y SNEPointNet++ [26] un *recall* superior al 92 %. Ambos segmentan grietas o desprendimientos en un elemento estructural escaneado de cerca, y evalúan sobre datos del mismo sitio. Liu et al. [28] reportan un IoU de 65,5 % para grietas en túneles con LiDAR. Zahs et al. [7] logran una exactitud global de 92–95 %, pero clasifican edificaciones completas, no cada punto. Aquí, en cambio, se segmenta punto a punto una escena aérea completa con cinco clases y se evalúa en una zona geográfica distinta a las de entrenamiento. Además, las etiquetas del dataset se obtuvieron proyectando a 3D anotaciones 2D hechas en uno de cada diez fotogramas, lo que introduce ruido en los bordes. En ese contexto, un IoU de 0,749 para la clase dañada es un resultado competitivo, y la mejora de 0,15 sobre PointNet++ (0,595) es relevante para una aplicación de triaje post-desastre.

5.3. Estabilidad de los resultados

La variabilidad entre semillas (Tabla 8) es tan importante como el promedio. Los modelos preentrenados alcanzan su mejor validación muy pronto y luego se sobreajustan, así que el checkpoint elegido depende del azar del entrenamiento. Además, el área de validación, con seis nubes, no siempre distingue un checkpoint bueno de uno re-

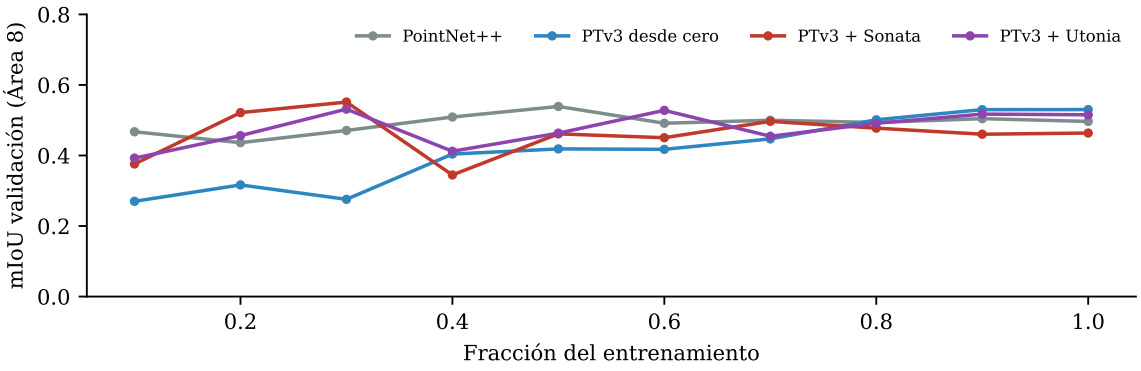


Figura 6: mIoU de validación (Área 8) durante el entrenamiento, semilla 0 de cada modelo. El eje horizontal es la fracción del entrenamiento completada (40 épocas para PointNet++ y 30 para los modelos PTv3).

Tabla 9: Comparación con los modelos de referencia de 3DAeroRelief en el Área 2 (IoU por clase, mIoU y mAcc). Las cinco primeras filas son los valores reportados en [6]; las de este trabajo son la media de sus semillas, a la resolución original y con TTA.

Método	Edif. dañada	Edif. sin daño	Vía	Árbol	Fondo	mIoU	mAcc
PTv1 [19]	0,511	0,154	0,300	0,191	0,385	0,308	0,461
PTv2 [20]	0,002	0,253	0,486	0,037	0,501	0,256	0,394
PTv3 [21]	0,761	0,311	0,047	0,400	0,773	0,458	0,551
FPT [23]	0,140	0,030	0,000	0,000	0,410	0,110	0,220
OA-CNNs [22]	0,582	0,196	0,000	0,296	0,228	0,260	0,440
PointNet++	0,595	0,133	0,044	0,608	0,799	0,436	0,606
PTv3 desde cero	0,731	0,172	0,042	0,572	0,716	0,447	0,637
PTv3 + Sonata	0,745	0,269	0,068	0,643	0,799	0,505	0,691
PTv3 + Utonia	0,749	0,310	0,222	0,639	0,807	0,545	0,737

Tabla 10: Exactitud por nube del Área 2 (semilla 0, puntos submuestreados, con TTA).

Nube	PointNet++	PTv3 + Utonia	Dif.
pp1	0,72	0,67	−0,05
pp2	0,66	0,75	+0,09
pp3	0,78	0,82	+0,04
pp4	0,82	0,80	−0,02
pp5	0,61	0,53	−0,08
pp6	0,60	0,92	+0,33
pp7	0,79	0,93	+0,15
pp8	0,88	0,96	+0,09

gular. Las diferencias pequeñas entre modelos, como entre PTv3 desde cero y PointNet++, no son concluyentes. La ventaja de PTv3 + Utonia es robusta: todas sus semillas superan a todas las de los modelos entrenados desde cero. La de PTv3 + Sonata no lo es: alguna de sus semillas queda por debajo de la mejor semilla de un modelo entrenado desde cero. La evaluación por nube (Tabla 10) confirma que el promedio esconde diferencias grandes entre escenas. También muestra que la resolución de evaluación cambia las métricas, porque las nubes densas pesan más: por eso este trabajo reporta ambas resoluciones y compara con el benchmark a la resolución original.

5.4. Análisis de errores

El modelo reconoce bien la edificación dañada, que es la clase prioritaria para el triaje. Su error principal es el opuesto: marcar como dañada parte de las edificaciones sanas. A 0,1 m de resolución, un techo intacto y uno con daño leve (tejas levantadas, lonas) son geométricamente parecidos, y en el área de prueba la edificación dañada es mayoritaria, lo que sesga la decisión hacia esa clase. Para una herramienta de inspección este tipo de error es preferible al contrario, porque genera falsas alarmas que un inspector descarta, en lugar de daños que pasen inadvertidos. La vía se confunde con el fondo porque ambas son superficies planas a ras de suelo, y la vía representa solo el 0,6 % de los puntos de prueba, así que su IoU no permite conclusiones.

5.5. Limitaciones

- El área de prueba tiene solo 8 nubes de una misma zona. Los resultados varían entre nubes y entre semillas, y los modelos no tienen todos el mismo número de semillas.
- La comparación con el benchmark es indicativa, porque el preprocesamiento no es idéntico.
- Por el presupuesto de cómputo (una GPU de 8 GB), se usaron las versiones pequeñas de los codificadores preentrenados y 30 épocas para PTv3.

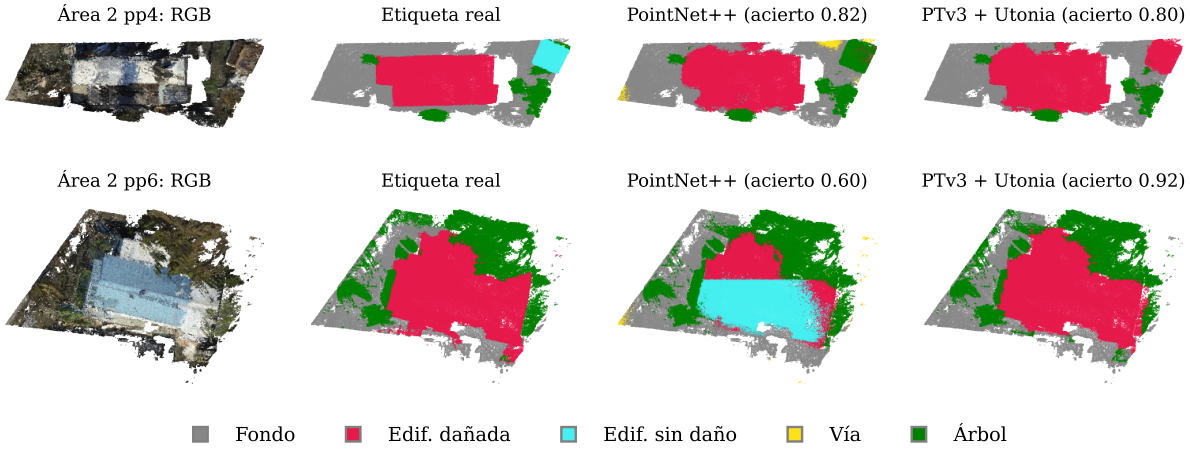


Figura 7: Ejemplos cualitativos sobre el Área 2 (vista superior): color RGB, etiqueta real, predicción de PointNet++ y predicción de PTv3 + Utonia (semilla 0).

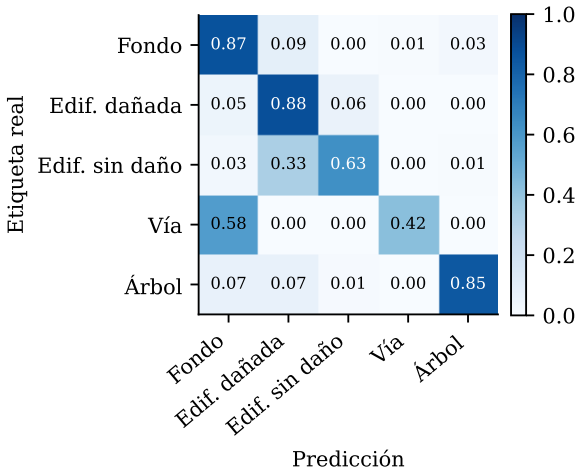


Figura 8: Matriz de confusión normalizada por fila de PTv3 + Utonia en el Área 2 (semilla 0, resolución original, con TTA).

6. Conclusiones y trabajo futuro

Este trabajo abordó la detección automática de edificaciones dañadas en nubes de puntos de dron, desde la revisión de la literatura hasta una herramienta utilizable. Las conclusiones principales son:

- **Es viable.** En una zona que el modelo nunca vio, PTv3 + Utonia identifica la edificación dañada con un IoU de 0,749 a la resolución original, y supera, de forma indicativa, los resultados publicados para el benchmark.
- **El preentrenamiento importa más que la arquitectura.** Un PTv3 entrenado desde cero queda al nivel de PointNet++ (mIoU 0,447 frente a 0,436), mientras que el mismo PTv3 preentrenado lo supera con claridad: PTv3 + Sonata 0,505, PTv3 + Utonia 0,545. Sin embargo, el ajuste de estos modelos es inestable: con la misma configuración, distintas semillas difieren en varios puntos de mIoU, y la validación no siempre lo anticipa.
- **Revisar los datos es parte del método.** La orientación arbitraria y la escala inconsistente no estaban documentadas; corregirlas fue necesario para que rasgos como la altura tengan significado.
- **La evaluación debe ser cuidadosa.** Las métricas cambian con la resolución de evaluación y varían mucho entre nubes y entre semillas, así que reportar una sola cifra puede ser engañoso.
- **Del modelo a la herramienta.** La aplicación desarrollada reutiliza el mismo código del experimento y funciona sin conexión, lo que la acerca a un uso real en campo.

Como trabajo futuro se proponen cinco líneas: (i) estabilizar el ajuste de los modelos preentrenados, por ejemplo con tasas de aprendizaje menores, congelando parte del codificador o promediando pesos de varias épocas; (ii) mejorar la distinción entre edificaciones sanas y con daño leve, por ejemplo con mayor resolución o rasgos geométricos de rugosidad del techo [26]; (iii) usar codificadores preentrenados más grandes y conjuntos de datos de otras zonas y tipos de desastre; (iv) incorporar escalas de daño más finas que la clasificación binaria; y (v) validar la herramienta con usuarios del área de gestión del riesgo.

Referencias

[1] F. Hake, P. Lippmann, H. Alkhatib, V. Oettel, I. Neumann, Automated damage detection for port structures

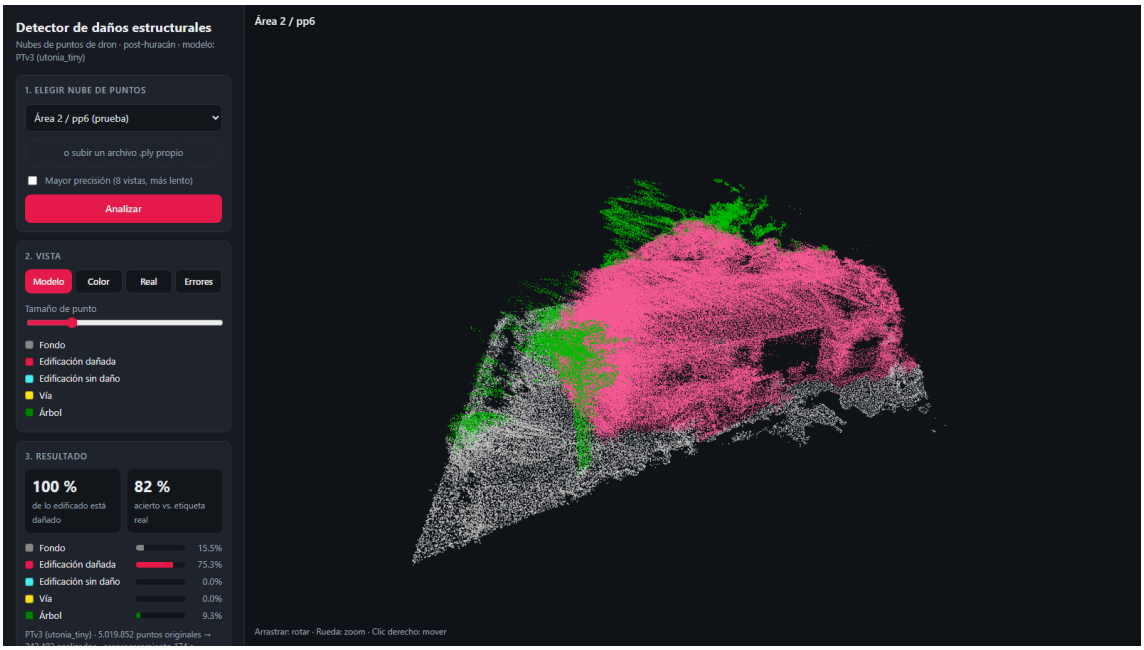


Figura 9: Interfaz de la herramienta analizando una nube de prueba (Área 2). A la izquierda, la selección de la nube, los modos de vista y el resumen; a la derecha, el visor 3D con la predicción del modelo: edificación dañada (rojo), edificación sin daño (celeste), vegetación (verde), vías (amarillo) y fondo (gris).

using machine learning algorithms in heightfields, Applied Geomatics 15(2) (2023) 349–357. https://doi.org/10.1007/s12518-023-00493-z

[2] S. Chen, G. Fan, J. Li, H. Hao, Automatic complex concrete crack detection and quantification based on point clouds and deep learning, Engineering Structures 327 (2025) 119635. https://doi.org/10.1016/j.engstruct.2025.119635

[3] X. Chen, J. Li, S. Huang, H. Cui, P. Liu, Q. Sun, An automatic concrete crack-detection method fusing point clouds and images based on improved Otsu's algorithm, Sensors 21(5) (2021) 1581. https://doi.org/10.3390/s21051581

[4] S. Dorafshan, R. J. Thomas, M. Maguire, SDNET2018: An annotated image dataset for non-contact concrete crack detection using deep convolutional neural networks, Data in Brief 21 (2018) 1664–1668. https://doi.org/10.1016/j.dib.2018.11.015

[5] Y. Lou, S. Meng, Y. Zhou, Deep learning-based three-dimensional crack damage detection method using point clouds without color information, Structural Health Monitoring 24(2) (2025) 657–675. https://doi.org/10.1177/14759217241236929

[6] N. Le, E. Karimi, M. Rahnemoonfar, 3DAeroRelief: The first 3D benchmark UAV dataset for post-disaster assessment, Scientific Data 13 (2026) 1305. https://doi.org/10.1038/s41597-026-07341-w

[7] V. Zahs, K. Anders, J. Kohns, A. Stark, B. Höfle, Classification of structural building damage grades from multi-temporal photogrammetric point clouds using a machine learning model trained on virtual laser scanning data, International Journal of Applied Earth Observation and Geoinformation 122 (2023) 103406. https://doi.org/10.1016/j.jag.2023.103406

[8] Y. Jing, J.-X. Zhong, B. Sheil, S. Acikgoz, Anomaly detection of cracks in synthetic masonry arch bridge point clouds using fast point feature histograms and PatchCore, Automation in Construction 168 (2024) 105766. https://doi.org/10.1016/j.autcon.2024.105766

[9] Y. Jing, W. Lin, B. Sheil, S. Acikgoz, 3D multimodal feature for infrastructure anomaly detection, Automation in Construction 178 (2025) 106388. https://doi.org/10.1016/j.autcon.2025.106388

[10] D. Dais, I. E. Bal, E. Smyrou, V. Sarhosis, Automatic crack classification and segmentation on masonry surfaces using convolutional neural networks and transfer learning, Automation in Construction 125 (2021) 103606. https://doi.org/10.1016/j.autcon.2021.103606

[11] Y. Jing, B. Sheil, S. Acikgoz, Segmentation of large-scale masonry arch bridge point clouds with a synthetic simulator and the BridgeNet neural network, Automation in Construction 142 (2022) 104459. https://doi.org/10.1016/j.autcon.2022.104459

[12] L. J. Sánchez-Aparicio, R. Santamaría-Maestro, P. Sanz-Honrado, P. Villanueva-Llauradó, et al., A holistic solution for supporting the diagnosis of historic constructions from 3D point clouds, Remote Sensing 17(12) (2025) 2018. https://doi.org/10.3390/rs17122018

[13] P. Sanz-Honrado, R. Santamaría-Maestro, L. J. Sánchez-Aparicio, Seg4D: An open-source solution for supporting the diagnosis of historic constructions using 3D point clouds – A case study application, The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences XLVIII-2/W12-2026 (2026) 439–446. https://doi.org/10.5194/isprs-archives-XLVIII-2-W12-2026-439-2026

[14] D. Loverdos, V. Sarhosis, Pixel-level block classification and crack detection from 3D reconstruction models of masonry structures using convolutional neural networks, Engineering Structures 310 (2024) 118113. https://doi.org/10.1016/j.engstruct.2024.118113

[15] Z. Ye, L. Lovell, A. Faramarzi, J. Ninić, SAM-based instance segmentation models for the automation of structural damage detection, Advanced Engineering Informatics 62 (2024) 102826. https://doi.org/10.1016/j.aei.2024.102826

[16] C. R. Qi, H. Su, K. Mo, L. J. Guibas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 77–85. https://doi.org/10.1109/CVPR.2017.16

[17] C. R. Qi, L. Yi, H. Su, L. J. Guibas, PointNet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5099–5108. https://arxiv.org/abs/1706.02413

[18] G. Qian, Y. Li, H. Peng, J. Mai, H. A. A. K. Hammoud, M. Elho-

seiny, B. Ghanem, PointNeXt: Revisiting PointNet++ with improved training and scaling strategies, in: Advances in Neural Information Processing Systems 35 (NeurIPS), 2022, pp. 23192–23204. https://doi.org/10.52202/068431-1685

[19] H. Zhao, L. Jiang, J. Jia, P. Torr, V. Koltun, Point Transformer, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 16239–16248. https://doi.org/10.1109/ICCV48922.2021.01595

[20] X. Wu, Y. Lao, L. Jiang, X. Liu, H. Zhao, Point Transformer V2: Grouped vector attention and partition-based pooling, in: Advances in Neural Information Processing Systems 35 (NeurIPS), 2022, pp. 33330–33342. https://doi.org/10.52202/068431-2415

[21] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, H. Zhao, Point Transformer V3: Simpler, faster, stronger, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 4840–4851. https://doi.org/10.1109/CVPR52733.2024.00463

[22] B. Peng, X. Wu, L. Jiang, Y. Chen, H. Zhao, Z. Tian, J. Jia, OA-CNNs: Omni-adaptive sparse CNNs for 3D semantic segmentation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 21305–21315. https://doi.org/10.1109/CVPR52733.2024.02013

[23] C. Park, Y. Jeong, M. Cho, J. Park, Fast Point Transformer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 16928–16937. https://doi.org/10.1109/CVPR52688.2022.01644

[24] M. Rahnemoonfar, T. Chowdhury, R. Murphy, RescueNet: A high resolution UAV semantic segmentation dataset for natural disaster damage assessment, Scientific Data 10 (2023) 913. https://doi.org/10.1038/s41597-023-02799-4

[25] Q. Hu, B. Yang, S. Khalid, W. Xiao, N. Trigoni, A. Markham, SensatUrban: Learning semantics from urban-scale photogrammetric point clouds, International Journal of Computer Vision 130(2) (2022) 316–343. https://doi.org/10.1007/s11263-021-01554-9

[26] N. Bolourian, M. Nasrollahi, F. Bahreini, A. Hammad, Point cloud-based concrete surface defect semantic segmentation, Journal of Computing in Civil Engineering 37(2) (2023) 04022056. https://doi.org/10.1061/JCCEE5.CPENG-5009

[27] J. Fan, W. Song, J. Zhang, S. Sun, G. Jia, G. Jin, PAN: Improved PointNet++ for pavement crack information extraction, Electronics 13(16) (2024) 3340. https://doi.org/10.3390/electronics13163340

[28] S. Liu, K. Isobe, J. Si, D. Li, D. Ren, A multi-module deep learning framework with graph-based network and crack attention for tunnel lining crack segmentation from LiDAR point cloud, Construction and Building Materials 494 (2025) 143383. https://doi.org/10.1016/j.conbuildmat.2025.143383

[29] H. Xu, M. Wang, C. Liu, Y. Guo, Z. Gao, C. Xie, Tunnel crack assessment using simultaneous localization and mapping (SLAM) and deep learning segmentation, Automation in Construction 171 (2025) 105977. https://doi.org/10.1016/j.autcon.2025.105977

[30] X. Yang, E. del Rey Castillo, Y. Zou, L. Wotherspoon, Semantic segmentation of bridge point clouds with a synthetic data augmentation strategy and graph-structured deep metric learning, Automation in Construction 150 (2023) 104838. https://doi.org/10.1016/j.autcon.2023.104838

[31] S. Shah, J. Shan, P. Li, UAV-based 3D segmentation and structural damage assessment of regional buildings, Journal of Civil Structural Health Monitoring 15(8) (2025) 4081–4105. https://doi.org/10.1007/s13349-025-01030-9

[32] X. Wu, D. DeTone, D. Frost, T. Shen, C. Xie, N. Yang, J. Engel, R. Newcombe, H. Zhao, J. Straub, Sonata: Self-supervised learning of reliable point representations, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 22193–22204. https://doi.org/10.1109/CVPR52734.2025.02067

[33] Y. Zhang, X. Wu, Y. Yang, X. Fan, H. Li, Y. Zhang, Z. Huang, N. Wang, H. Zhao, Utonia: Toward one encoder for all point clouds, in: International Conference on Machine Learning (ICML), 2026. https://arxiv.org/abs/2603.03283